

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..51

BEST PRACTICES

 Copy as Markdown

Best practices

Imports

The generated test tools file re-exports the test helpers, so importing `serverpod_test` as well brings in the same names twice and adds noise for no gain.

Don't

```
import 'test_tools/serverpod_test_tools.dart';  
// Don't import `serverpod_test` directly.  
import 'package:serverpod_test/serverpod_test.dart'; ❌
```

Do

```
// The generated file carries the test helpers and your endpoints.  
import 'test_tools/serverpod_test_tools.dart'; ✅
```

Serverpod's own types are a separate matter. The generated file does not re-export them, so a test that uses `Session`, `Scope`, `Constant`, `ServerpodRunMode`, `ExperimentalFeatures`, or a Serverpod exception type needs `package:serverpod/serverpod.dart` as well. Your own models come from your project's generated protocol.

Database clean up

Unless configured otherwise, by default `withServerpod` does all database operations inside a transaction that is rolled back after each `test` (see [the configuration options](#) for more info on this behavior).

Don't

```
withServerpod('Given ProductsEndpoint', (sessionBuilder, endpoints) {
```



Ask AI

```

var session = sessionBuilder.build();

setUp(() async {
  await Product.db.insertRow(session, Product(name: 'Apple', price: 10));
});

tearDown(() async {
  await Product.db.deleteWhere( ✗ // Unnecessary clean up
    session,
    where: (_) ⇒ Constant.bool(true),
  );
});

// ...
});

```

Do

```

withServerpod('Given ProductsEndpoint', (sessionBuilder, endpoints) {
  var session = sessionBuilder.build();

  setUp(() async {
    await Product.db.insertRow(session, Product(name: 'Apple', price: 10));
  });

  ✓ // Clean up can be omitted since the transaction is rolled back after each by default

  // ...
});

```

Calling endpoints

While it's technically possible to instantiate an endpoint class and call its methods directly with a `ServerpodSession`, it's advised that you do not. The reason is that lifecycle events and validation that should happen before or after an endpoint method is called is taken care of by the framework. Calling endpoint methods directly would circumvent that and the code would not behave like production code. Using the test tools guarantees that the way endpoints behave during tests is the same as in production.

Don't

```

void main() {
  // ✗ Don't instantiate endpoints directly
  var greetingEndpoint = GreetingEndpoint();

  withServerpod('Given Greeting endpoint', (
    sessionBuilder,
    _ /* not using the provided endpoints */,
  ) {
    var session = sessionBuilder.build();

```

```
test('when calling `hello` then should return greeting', () async {
  // ❌ Don't call an endpoint method directly on the endpoint class.
  final greeting = await greetingEndpoint.hello(session, 'Bob');
  expect(greeting.message, 'Hello Bob');
});
});
}
```

Do

```
void main() {
  withServerpod('Given Greeting endpoint', (sessionBuilder, endpoints) {
    test('when calling `hello` then should return greeting', () async {
      // ✅ Use the provided `endpoints` to call the endpoint that should be tested.
      final greeting = await endpoints.greeting.hello(sessionBuilder, 'Bob');
      expect(greeting.message, 'Hello Bob');
    });
  });
}
```

Unit and integration tests

It is significantly easier to navigate a project if the different types of tests are clearly separated.

Don't

❌ Mix different types of tests together.

Do

✅ Have a clear structure for the different types of test. Serverpod recommends the following two folders in the `server`:

- `test/unit`: Unit tests.
- `test/integration`: Tests for endpoints or business logic modules using the `withServerpod` helper.

Related

- [Advanced examples](#): patterns for streams, future calls, and business logic.
- [Configuration](#): the options `withServerpod` accepts.
- [Deploy to Serverpod Cloud](#): ship the server your tests cover.